Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине «Современные платформы программирования»

Выполнил:
Студент 3 курса
Группы ПО-12
Пашкевич В. В.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы: *закрепить навыки объектно-ориентированного программирования на языке Python.*

Задание 1

Реализовать простой класс

Вариант 4

Прямоугольник, заданный длинами двух сторон – Предусмотреть возможность определения площади и периметра, а также логические методы, определяющие, является ли прямоугольник квадратом и существует ли такой прямоугольник. Конструктор должен позволять создавать объекты с начальной инициализацией. Переопределить метод __eq__, выполняющий сравнение объектов данного типа.

Решение

```
class Rectangle:
    def __init__(self, a: float, b: float):
        self.a = a
        self.b = b

    @property
    def a(self):
        return self._a

    @a.setter
    def a(self, value):
        if not isinstance(value, (int, float)):
            raise TypeError("Длина стороны должна быть числом")
        self._a = value

    @property
    def b(self):
        return self._b

    @b.setter
    def b(self, value):
        if not isinstance(value, (int, float)):
            raise TypeError("Длина стороны должна быть числом")
        self._b = value

    def exists(self) -> bool:
        return self.a > 0 and self.b > 0

    def area(self):
        if self.exists():
            return self.a * self.b
        return 0

    def perimeter(self):
        if self.exists():
            return 2 * (self.a + self.b)
        return 0

    def is_square(self) -> bool:
        return self.exists() and self.a == self.b
```

```python
    def __str__(self):
        if self.exists():
            return (f"Прямоугольник со сторонами {self.a} и {self.b}. "
                    f"Площадь: {self.area()}, Периметр: {self.perimeter()}, "
                    f"Квадрат: {'да' if self.is_square() else 'нет'}")
        return f"Прямоугольник со сторонами {self.a} и {self.b} не
существует"

    def __eq__(self, other):
        if not isinstance(other, Rectangle):
            return False
        return self.a == other.a and self.b == other.b


# Пример использования
r1 = Rectangle(5, 10)
r2 = Rectangle(5, 10)
r3 = Rectangle(7, 7)
r4 = Rectangle(-2, 4)

print(r1)
print(r3)
print(r4)

print("r1 == r2:", r1 == r2)
print("r1 == r3:", r1 == r3)
print("Площадь r1:", r1.area())
print("Периметр r1:", r1.perimeter())
print("r3 является квадратом:", r3.is_square())
print("r4 существует:", r4.exists())
```

```
Прямоугольник со сторонами 5 и 10. Площадь: 50, Периметр: 30, Квадрат: нет
Прямоугольник со сторонами 7 и 7. Площадь: 49, Периметр: 28, Квадрат: да
Прямоугольник со сторонами -2 и 4 не существует
r1 == r2: True
r1 == r3: False
Площадь r1: 50
Периметр r1: 30
r3 является квадратом: True
r4 существует: False
```

Задание 2

Построить модель программной системы с применением отношений (обобщения, агрегации, ассоциации, реализации) между классами. Задать атрибуты и методы классов. Реализовать (если необходимо) дополнительные классы. Продемонстрировать работу разработанной системы

Вариант 4

Система Интернет-магазин. Администратор добавляет информацию о Товаре. Клиент делает и оплачивает Заказ на Товары. Администратор регистрирует Продажу и может занести неплательщиков в «черный список».

Решение

```
from abc import ABC, abstractmethod


class Payable(ABC):
    @abstractmethod
    def pay(self):
        """Оплатить заказ."""


class User:
    def __init__(self, name: str):
        self.name = name

    def __str__(self):
        return f"Пользователь: {self.name}"


class Admin(User):
    def add_product(self, shop, product):
        shop.products.append(product)
        print(f"Администратор {self.name} добавил товар: {product.name}")

    def register_sale(self, shop, order):
        if order.is_paid:
            shop.sales.append(Sale(order))
            print(f"Администратор {self.name} зарегистрировал продажу по
заказу №{order.order_id}")
        else:
            print(f"Заказ №{order.order_id} не оплачен. Продажа не
зарегистрирована.")

    def add_to_blacklist(self, shop, client):
        shop.blacklist.add(client)
        print(f"Клиент {client.name} добавлен в черный список.")


class Client(User):
    def make_order(self, shop, products):
        if self in shop.blacklist:
            print(f"Клиент {self.name} находится в черном списке и не может
оформить заказ.")
            return None

        order = Order(self, products)
        shop.orders.append(order)
        print(f"Клиент {self.name} оформил заказ №{order.order_id}")
        return order


class Product:
    def __init__(self, product_id: int, name: str, price: float):
        self.product_id = product_id
        self.name = name
        self.price = price

    @property
    def price(self):
        return self._price

    @price.setter
    def price(self, value):
```

```
        if value < 0:
            raise ValueError("Цена не может быть отрицательной")
        self._price = value

    def __str__(self):
        return f"Товар(id={self.product_id}, название='{self.name}',
цена={self.price})"

    def __eq__(self, other):
        if not isinstance(other, Product):
            return False
        return self.product_id == other.product_id


class Order(Payable):
    order_counter = 1

    def __init__(self, client, products):
        self.order_id = Order.order_counter
        Order.order_counter += 1
        self.client = client
        self.products = products[:]
        self.is_paid = False

    def total_amount(self):
        return sum(product.price for product in self.products)

    def pay(self):
        self.is_paid = True
        print(f"Заказ №{self.order_id} оплачен. Сумма:
{self.total_amount()}")

    def __str__(self):
        product_names = ", ".join(product.name for product in self.products)
        return (
            f"Заказ №{self.order_id}: клиент={self.client.name}, "
            f"товары=[{product_names}], сумма={self.total_amount()}, "
            f"оплачен={'да' if self.is_paid else 'нет'}"
        )

    def __eq__(self, other):
        if not isinstance(other, Order):
            return False
        return self.order_id == other.order_id


class Sale:
    sale_counter = 1

    def __init__(self, order):
        self.sale_id = Sale.sale_counter
        Sale.sale_counter += 1
        self.order = order

    def __str__(self):
        return f"Продажа №{self.sale_id} по заказу №{self.order.order_id}"

    def __eq__(self, other):
        if not isinstance(other, Sale):
            return False
        return self.sale_id == other.sale_id


class OnlineStore:
```

```python
    def __init__(self, name: str):
        self.name = name
        self.products = []
        self.orders = []
        self.sales = []
        self.blacklist = set()

    def show_products(self):
        print("\nСписок товаров:")
        if not self.products:
            print("Товаров нет.")
        for product in self.products:
            print(product)

    def show_orders(self):
        print("\nСписок заказов:")
        if not self.orders:
            print("Заказов нет.")
        for order in self.orders:
            print(order)

    def show_sales(self):
        print("\nСписок продаж:")
        if not self.sales:
            print("Продаж нет.")
        for sale in self.sales:
            print(sale)

    def show_blacklist(self):
        print("\nЧерный список:")
        if not self.blacklist:
            print("Черный список пуст.")
        for client in self.blacklist:
            print(client.name)

    def __str__(self):
        return f"Интернет-магазин: {self.name}"

    def __eq__(self, other):
        if not isinstance(other, OnlineStore):
            return False
        return self.name == other.name


def main():
    shop = OnlineStore("MyShop")

    admin = Admin("Иван")
    client1 = Client("Алексей")
    client2 = Client("Мария")

    product1 = Product(1, "Ноутбук", 50000)
    product2 = Product(2, "Мышь", 1500)
    product3 = Product(3, "Клавиатура", 3000)

    admin.add_product(shop, product1)
    admin.add_product(shop, product2)
    admin.add_product(shop, product3)

    shop.show_products()

    order1 = client1.make_order(shop, [product1, product2])
    if order1:
        print(order1)
```

```
        order1.pay()
        admin.register_sale(shop, order1)

    order2 = client2.make_order(shop, [product3])
    if order2:
        print(order2)
        admin.register_sale(shop, order2)
        admin.add_to_blacklist(shop, client2)

    shop.show_orders()
    shop.show_sales()
    shop.show_blacklist()

    client2.make_order(shop, [product2])


if __name__ == "__main__":
    main()
```

```
Администратор Иван добавил товар: Ноутбук
Администратор Иван добавил товар: Мышь
Администратор Иван добавил товар: Клавиатура

Список товаров:
Товар(id=1, название='Ноутбук', цена=50000)
Товар(id=2, название='Мышь', цена=1500)
Товар(id=3, название='Клавиатура', цена=3000)
Клиент Алексей оформил заказ №1
Заказ №1: клиент=Алексей, товары=[Ноутбук, Мышь], сумма=51500, оплачен=нет
Заказ №1 оплачен. Сумма: 51500
Администратор Иван зарегистрировал продажу по заказу №1
Клиент Мария оформил заказ №2
Заказ №2: клиент=Мария, товары=[Клавиатура], сумма=3000, оплачен=нет
Заказ №2 не оплачен. Продажа не зарегистрирована.
Клиент Мария добавлен в черный список.

Список заказов:
Заказ №1: клиент=Алексей, товары=[Ноутбук, Мышь], сумма=51500, оплачен=да
Заказ №2: клиент=Мария, товары=[Клавиатура], сумма=3000, оплачен=нет

Список продаж:
Продажа №1 по заказу №1

Черный список:
Мария
Клиент Мария находится в черном списке и не может оформить заказ.
```

Вывод: *закрепил навыки объектно-ориентированного программирования на языке Python.*